

# 1. Docker Basics

## **docker --version**

Check Docker version.

## **docker info**

Show system-wide Docker settings.

## **docker login**

Login to Docker Hub / private registry.

---

# 2. Docker Images

## **docker pull nginx**

Pull image from Docker Hub.

## **docker images**

List local images.

## **docker build -t app:v1 .**

Build image from Dockerfile.

## **docker rmi image-id**

Remove an image.

## **docker tag app:v1 repo/app:v1**

Tag image for push.

## **docker push repo/app:v1**

Push image to registry.

## **docker history nginx**

Show how an image was built layer-by-layer.

---

## 3. Docker Containers

**docker run nginx**

Run a container.

**docker run -d nginx**

Run in background.

**docker run -p 8080:80 nginx**

Map container port → host port.

**docker run --name web nginx**

Run container with name.

**docker ps**

List running containers.

**docker ps -a**

List all containers.

**docker stop container-id**

Stop container.

**docker start container-id**

Start container.

**docker restart container-id**

Restart container.

**docker rm container-id**

Remove container.

**docker rm -f container-id**

Force remove container.

---

## 4. Docker Logs & Debugging

**docker logs container-id**

Show logs.

**docker logs -f container-id**

Follow live logs.

**docker inspect container-id**

Complete container info (IP, mounts, env, config).

**docker exec -it container-id bash**

Enter container shell.

**docker exec -it container-id sh**

Enter Alpine shell.

---

## 5. Docker Volumes

**docker volume create data-vol**

Create volume.

**docker volume ls**

List volumes.

**docker volume inspect data-vol**

Inspect volume.

**docker run -v data-vol:/app nginx**

Attach volume to container.

**docker volume rm data-vol**

Delete volume.

---

## 6. Docker Bind Mounts

**docker run -v /host/data:/container/data nginx**

Bind host directory into container.

**Used for:**

- ✓ Live code changes
- ✓ Sharing logs
- ✓ Local dev

---

## 7. Docker Networks

**docker network ls**

List networks.

**docker network create app-net**

Create custom network.

**docker run --network app-net nginx**

Run container on a network.

**docker network inspect app-net**

Network info.

---

## 8. Docker Compose

**docker-compose up**

Start all services.

**docker-compose up -d**

Start in background.

**docker-compose down**

Stop + remove containers, network.

**docker-compose logs**

View logs from all services.

**docker-compose ps**

List compose containers.

**docker-compose build**

Build images via Compose.

---

## 9. Docker Cleanup

### **docker system prune**

Remove unused containers, images, networks.

### **docker system prune -a**

Remove ALL unused images (aggressive).

### **docker container prune**

Remove stopped containers.

### **docker image prune**

Remove unused images only.

---

## 10. Docker Resource Stats

### **docker stats**

Live container CPU/memory/IO usage.

### **docker top container-id**

Show container processes.

### **docker events**

Real-time Docker event stream.

---

## 11. Docker Copy Commands

### **docker cp file.txt container:/app/**

Copy file into container.

### **docker cp container:/app/logs .**

Copy files from container → host.

---

## 12. Dockerfile Commands (DevOps Essentials)

### **FROM**

Base image.

### **RUN**

Run a command.

### **COPY src dest**

Copy files.

### **ADD**

Copy + extract (not recommended).

### **CMD**

Default command to run.

### **ENTRYPOINT**

Main executable.

### **EXPOSE 80**

Expose port.

### **WORKDIR /app**

Set working directory.

### **ENV APP\_ENV=prod**

Set environment variable.

### **VOLUME /data**

Create mount point.

---

## 13. Docker Image Management

### **docker save -o image.tar image-name**

Save image to tarball.

### **docker load -i image.tar**

Load image from tarball.

```
docker export container-id > file.tar
```

Export container filesystem.

```
docker import file.tar newimage
```

Import as image.

---

## 14. Docker Health Checks

```
docker inspect --format='{{json .State.Health}}' container-id
```

Check health of a container.

**Add in Dockerfile:**

```
HEALTHCHECK CMD curl -f http://localhost/ || exit 1
```

---

## 15. Docker Restart Policies

```
docker run --restart=always nginx
```

Always restart container if crashed.

Policies:

- no
  - always
  - on-failure
  - unless-stopped
- 

## 16. Working with Docker Registries

```
docker login registry.example.com
```

Login to private registry.

```
docker tag app registry.example.com/app
```

Tag image for registry.

**docker push registry.example.com/app**

Push to private registry.

---

## 17. Docker Security Commands

**docker scan image**

Scan image for vulnerabilities.

**docker diff container-id**

Show file changes in container filesystem.

**docker inspect --format='{{.Config.User}}' container-id**

Check which user container runs as.

---

## 18. Multi-stage Builds

Used for small production images.

```
FROM golang:1.20 AS build
WORKDIR /app
RUN go build -o myapp
```

```
FROM alpine
COPY --from=build /app/myapp .
CMD ["/myapp"]
```

---

## 19. Docker Swarm (If used)

**docker swarm init**

Initialize swarm cluster.

**docker node ls**

List nodes.

**docker service create --replicas=3 nginx**

Create service.

---



## 20. Docker System Diagnostics

**docker inspect <container>**

Deep details on container.

**docker info**

Docker engine system info.

**docker logs <container>**

Debugging.

---

## 21. Docker Environment Variables

**docker run -e ENV=prod nginx**

Pass environment variables to a container.

**docker run --env-file env.list nginx**

Load variables from a file.

**Used for:**

- ✓ Config injection
  - ✓ Secrets (non-sensitive)
  - ✓ CI/CD pipelines
- 

## 22. Docker Limits & Resource Control

**docker run --cpus=1 nginx**

Limit CPU usage.

**docker run -m 512m nginx**

Limit container memory.

**docker run --memory-swap=1g nginx**

Set memory + swap limit.

**docker stats**

Monitor container resource usage.

---

## 23. Docker Events & Daemon Debugging

### **docker events**

Live tracking of Docker activities (crashes, pulls, restarts).

### **docker inspect --format '{{.State.Status}}' container-id**

Check container state.

---

## 24. Docker Multi-Architecture Builds

### **docker buildx create --use**

Enable buildx builder.

### **docker buildx build --platform linux/amd64,linux/arm64 -t app:multi .**

Build multi-architecture images.

Used for:

- ✓ ARM (M1/M2 Mac)
  - ✓ AWS Graviton
  - ✓ Cross-platform builds
- 

## 25. Docker Compose Advanced Commands

### **docker-compose config**

Validate and view final merged configuration.

### **docker-compose restart**

Restart services.

### **docker-compose pull**

Pull images defined in compose file.

### **docker-compose exec web bash**

Execute inside a running service.

### **docker-compose down --volumes**

Delete containers + volumes.

---

## 26. Docker Image Optimization

**docker system df**

Show size usage of images, containers.

**docker builder prune**

Remove old build cache.

**docker image prune -a**

Remove all unused images.

---

## 27. Docker Networking Advanced

**docker network create --driver bridge custom-net**

Create custom bridge network.

**docker network create --driver overlay swarm-net**

Create overlay network (Swarm/K8s).

**docker network connect custom-net container-id**

Attach container to network.

**docker network disconnect custom-net container-id**

Detach container from network.

---

## 28. Docker Credentials & Secret Management

**docker login --username user --password-stdin**

Login securely using stdin.

**docker secret create db-pass secret.txt**

Create secret (Swarm mode).

**docker secret ls**

List secrets.

---

## 29. Docker Container Backup & Restore

**`docker export container-id > container.tar`**

Export container filesystem.

**`docker import container.tar new-image`**

Import as image.

**`docker commit container-id new-image`**

Create image from running container (not recommended except for debugging).

---

## 30. Docker File Sharing & Permissions

**`docker run -v /host:/app --user 1000:1000 nginx`**

Run container with specific user.

**`docker inspect container-id --format '{{.Config.User}}'`**

Find running user.

---

## 31. Docker Entrypoint & CMD Deep Explanation

**`CMD ["npm", "start"]`**

Default command.

**`ENTRYPOINT ["python", "app.py"]`**

Always executes, cannot be overridden easily.

### **Combined:**

`ENTRYPOINT ["python"]`  
`CMD ["app.py"]`

Output:

`python app.py`

---

## 32. Docker Layer Caching

### **docker history app**

View all layers and sizes.

#### **Use caching rules:**

- Layers with fewer changes go at top
  - Install dependencies before copying code
- 

## 33. Docker Inspect Useful Output

### **docker inspect -f '{{.NetworkSettings.IPAddress}}' container-id**

Get container IP.

### **docker inspect -f '{{.Mounts}}' container-id**

Check mounted volumes.

### **docker inspect -f '{{.Config.Env}}' container-id**

List environment variables.

---

## 34. Docker Log Drivers

### **docker run --log-driver json-file nginx**

Default logging.

### **docker run --log-driver=syslog nginx**

Send logs to syslog.

### **docker inspect --format '{{.HostConfig.LogConfig}}' container-id**

View log driver.

---

## 35. Docker Restart Policies (Advanced)

**docker update --restart=always container-id**

Set restart policy on existing container.

**docker run --restart=on-failure:5 app**

Restart on failure up to 5 times.

---

## 36. Docker Container Health Monitoring

**Add in Dockerfile:**

```
HEALTHCHECK --interval=5s --timeout=3s \
  CMD curl -f http://localhost/ || exit 1
```

**Check via CLI:**

```
docker inspect container-id | grep Health -A5
```

---

## 37. Docker Build Arguments (ARG)

**Dockerfile:**

```
ARG version
FROM node:${version}
```

**Build:**

```
docker build --build-arg version=18 -t myapp .
```

Used for:

- ✓ Dynamic base images
  - ✓ Passing temporary build-time values
- 

## 38. Docker Compose Profiles

**docker-compose.yml:**

```
profiles:
  - dev
```

**Run:**

```
docker-compose --profile dev up
```

Used for:

- ✓ Different envs (dev/stage/prod)
  - ✓ Optional services
- 

## 39. Docker Swarm (More Advanced)

```
docker service ls
```

List services.

```
docker service ps service-name
```

Show tasks of a service.

```
docker stack deploy -c docker-compose.yml mystack
```

Deploy compose as stack.

---

## 40. Docker System Repair Commands

```
sudo systemctl restart docker
```

Restart Docker daemon.

```
journalctl -u docker -f
```

View Docker daemon logs.

```
dockerd --debug
```

Run Docker engine in debug mode.